

CS602C - Lab Assignment 3

Lab Report

Roll Number: cs26mt010

Student: Snehadip Ghoshal

Repository folder: lab3

Task 1 - Build Using make and Run jp2a

Objective

Download jp2a 1.0.6, configure it for a local installation, compile it using make, add its bin directory to PATH, verify the executable with which jp2a, and convert a JPEG image to ASCII. The assignment requires the ASCII output to be redirected to task1_lab3_jp2a.txt.

Commands executed

```
mkdir -p ~/mtech_user/cs26mt010/lab3/task1
cd ~/mtech_user/cs26mt010/lab3/task1
wget https://sourceforge.net/projects/jp2a/files/jp2a/1.0.6/jp2a-1.0.6.tar.gz/download -O jp2a-1.0.6.tar.gz
tar -xzf jp2a-1.0.6.tar.gz
cd jp2a-1.0.6
sudo apt update
sudo apt install make libjpeg-dev
./configure --prefix=$(pwd)
make
make install
ls -l bin/
echo 'export PATH="$PATH:$HOME/mtech_user/cs26mt010/lab3/task1/jp2a-1.0.6/bin"' >> ~/.bashrc
source ~/.bashrc
which jp2a
cd ~/mtech_user/cs26mt010/lab3/task1
jp2a kohli.jpg > task1_lab3_jp2a.txt
cat task1_lab3_jp2a.txt
```

Observed verification output

```
$ ls -l ~/mtech_user/cs26mt010/lab3/task1/jp2a-1.0.6/bin/jp2a
-rwxr-xr-x 1 snehadip snehadip 78184 Aug 29 11:14 /home/snehadip/mtech_user/cs26mt010/lab3/task1/jp2a-1.0.6/bin/jp2a

$ which jp2a
/home/snehadip/mtech_user/cs26mt010/lab3/task1/jp2a-1.0.6/bin/jp2a
```

Dependency issue encountered

```
configure: error: I need a working jpeglib; please specify --with-jpeg-lib=PATH

$ sudo apt install libjpeg-dev
```

Result

jp2a was successfully built and installed. The executable was added to PATH and verified with which jp2a. The selected image was kohli.jpg and the required ASCII output file was task1_lab3_jp2a.txt. The raw ASCII image itself is intentionally not reproduced here; the generated file is the submission artifact for this part.

Task 2(A) - Linked List Makefile

Files

```
linkedlist.c
linkedlist.h
use_ll_2.c
use_linkedlist.c
Makefile
```

Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra

all: list_check list_run

linkedlist.o: linkedlist.c linkedlist.h
■$(CC) $(CFLAGS) -c $< -o $@

list_check: use_ll_2.c linkedlist.o
■$(CC) $(CFLAGS) $^ -o $@

list_run: use_linkedlist.c linkedlist.o
■$(CC) $(CFLAGS) $^ -o $@

clean:
■rm -f list_check list_run *.o
```

Build output

```
$ make
gcc -Wall -Wextra -c linkedlist.c -o linkedlist.o
gcc -Wall -Wextra use_ll_2.c linkedlist.o -o list_check
gcc -Wall -Wextra use_linkedlist.c linkedlist.o -o list_run
```

Clean and rebuild output

```
$ make clean
rm -f list_check list_run *.o

$ make
gcc -Wall -Wextra -c linkedlist.c -o linkedlist.o
gcc -Wall -Wextra use_ll_2.c linkedlist.o -o list_check
gcc -Wall -Wextra use_linkedlist.c linkedlist.o -o list_run
```

Program output

```
$ ./list_check
A node: 42■A node: 9999
10041

$ ./list_run
booyah
A node: 10■A node: 20■A node: 30
60
```

Make features used

The Makefile uses variables CC and CFLAGS and the automatic variables \$<, \$@ and \$^. The default target all builds both executables, and clean removes the executables and object files.

Task 2(B) - Calculator Makefile

Files

```
main.c
addition.c
subtraction.c
multiplication.c
division.c
operations.h
README.txt
Makefile
```

Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra

all: calculator

calculator: main.o addition.o subtraction.o multiplication.o division.o
	■$(CC) $(CFLAGS) $^ -o $@

main.o: main.c operations.h
	■$(CC) $(CFLAGS) -c $< -o $@

addition.o: addition.c operations.h
	■$(CC) $(CFLAGS) -c $< -o $@

subtraction.o: subtraction.c operations.h
	■$(CC) $(CFLAGS) -c $< -o $@

multiplication.o: multiplication.c operations.h
	■$(CC) $(CFLAGS) -c $< -o $@

division.o: division.c operations.h
	■$(CC) $(CFLAGS) -c $< -o $@

clean:
	■rm -f calculator *.o
```

Test commands and outputs

```
$ make
[calculator executable produced]

$ ./calculator 10 + 5
The result of your operation is 15.

$ ./calculator 10 - 5
The result of your operation is 5.

$ ./calculator 10 '*' 5
The result of your operation is 50.

$ ./calculator 10 / 5
The result of your operation is 2.

$ make clean
rm -f calculator *.o

$ make clean && make
[calculator rebuilt successfully]

$ ./calculator 20 + 30
The result of your operation is 50.
```

Task 3(A) - GDB Debugger Practice

Compilation

```
$ gcc -g -std=c99 -Wall -c L3.c -o L3.o
$ gcc -g -std=c99 -Wall -c main.c -o main.o
$ gcc -g L3.o main.o -lm -o L3
```

GDB session output

```
(gdb) set logging file gdb.txt
(gdb) set logging on
Copying output to gdb.txt.
Copying debug output to gdb.txt.
(gdb) list main.c:20
15         printf("Enter b: ");
16         scanf("%d", &b);
17         printf("Enter c: ");
18         scanf("%d", &c);
19         quadratic(a, b, c, &r1, &r2);
20         printf("Roots are %3.2f and %3.2f\n", r1, r2);
21     }
(gdb) break main.c:19
Breakpoint 1 at 0x1395: file main.c, line 19.
(gdb) break main.c:20
Breakpoint 2 at 0x13b0: file main.c, line 20.
(gdb) run
Starting program: /home/snehadip/mtech_user/cs26mt010/lab3/task3_A/L3
Quadratic Program
Enter a: 1
Enter b: -5
Enter c: 6
Breakpoint 1, main () at main.c:19
19     quadratic(a, b, c, &r1, &r2);
(gdb) print a
$1 = 1
(gdb) print b
$2 = -5
(gdb) print c
$3 = 6
(gdb) print r1
$4 = 0
(gdb) print r2
$5 = 0
(gdb) print &r1
$6 = (float *) 0x7fffffffde90
(gdb) print &r2
$7 = (float *) 0x7fffffffde94
(gdb) step
quadratic (coeff1=1, coeff2=-5, coeff3=6, root1=0x7fffffffde90, root2=0x7fffffffde94) at L3.c:10
10     if (coeff1 == 0)
(gdb) step
19         *root1=(-coeff2+sqrt((coeff2*coeff2)-(4*coeff1*coeff3)))/(2*coeff1);
(gdb) break L3.c:21
Breakpoint 3 at 0x55555555204: file L3.c, line 22.
(gdb) continue
Continuing.
(gdb) print root1
$8 = (float *) 0x7fffffffde90
(gdb) print root2
$9 = (float *) 0x7fffffffde94
(gdb) print *root1
$10 = 0
(gdb) print *root2
$11 = 0
(gdb) x/x root1
0x7fffffffde90: 0x00000000
(gdb) x/x root2
0x7fffffffde94: 0x00000000
(gdb) info breakpoints
Num      Type             Disp Enb Address                  What
1        breakpoint       keep y   0x000055555555395 in main
                                   at main.c:19
        breakpoint already hit 1 time
2        breakpoint       keep y   0x0000555555553b0 in main
                                   at main.c:20
3        breakpoint       keep y   0x000055555555204 in quadratic at L3.c:22
(gdb) disable 1
(gdb) info breakpoints
Num      Type             Disp Enb Address                  What
```

```

1      breakpoint      keep n    0x000055555555395 in main
                                           at main.c:19
      breakpoint already hit 1 time
2      breakpoint      keep y    0x0000555555553b0 in main
                                           at main.c:20
3      breakpoint      keep y    0x000055555555204 in quadratic at L3.c:22
(gdb) enable 1
(gdb) delete 1
(gdb) info breakpoints
Num      Type          Disp Enb Address              What
2        breakpoint    keep y    0x0000555555553b0 in main
                                           at main.c:20
3        breakpoint    keep y    0x000055555555204 in quadratic at L3.c:22
(gdb) info stack
#0  quadratic (coeff1=1, coeff2=-5, coeff3=6,
    root1=0x7fffffffde90, root2=0x7fffffffde94) at L3.c:19
#1  0x0000555555553b0 in main () at main.c:19
(gdb) info frame
Stack level 0, frame at 0x7fffffffde80:
rip = 0x55555555209 in quadratic (L3.c:19);
    saved rip = 0x555555553b0
    called by frame at 0x7fffffffdeb0
    source language c.
    Arglist at 0x7fffffffde70, args: coeff1=1, coeff2=-5,
    coeff3=6, root1=0x7fffffffde90, root2=0x7fffffffde94
    Locals at 0x7fffffffde70, Previous frame's sp is 0x7fffffffde80
(gdb) continue
Continuing.
Breakpoint 2, main () at main.c:20
20      printf("Roots are %.2f and %.2f\n", r1, r2);
(gdb) info locals
a = 1
b = -5
c = 6
r1 = 3
r2 = 2
(gdb) display r1
1: r1 = 3
(gdb) display r2
2: r2 = 2
(gdb) quit

```

Generated file

GDB logging was enabled with set logging file gdb.txt and set logging on. The complete session was saved as gdb.txt in task3_A and is submitted along with this report.

Task 3(B)(1) - Selection Sort and Binary Search

Initial crash

```
$ gcc -g -std=c99 -Wall selection.c -o selection
selection.c: In function 'selection_sort':
selection.c:15:1: warning: control reaches end of non-void function [-Wreturn-type]

$ ./selection
unsorted list:
    100
     45
     89
     27
    317
     17
Segmentation fault
```

GDB evidence

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
... at selection.c:8
8         if ( A[j] > A[max] )

(gdb) backtrace
#0  selection_sort (A=0x7fffffffde70, size=6) at selection.c:8
#1  main () at selection.c:43

(gdb) list
3     int selection_sort(int A[], int size) {
4         int i, j, max, temp;
5         for (i = size-1; i > 0; i--) {
6             max = 0;
7             for (j = 1; i <= size; j++) {
8                 if ( A[j] > A[max] )
9                     max = j;
10            }
11            temp = A[max];
12            A[max] = A[i];

(gdb) info locals
i = 5
j = 1124
max = 23
temp = 21845
```

First correction

Original:
for (j = 1; i <= size; j++) {

Corrected:
for (j = 1; j <= i; j++) {

Binary-search infinite loop

```
(gdb) run
... test if 42 is in the array...
^C
Program received signal SIGINT, Interrupt.
binary_search (item=42, A=0x7fffffffde70, size=6) at selection.c:28
28         if (item < A[mid])

(gdb) backtrace
#0  binary_search (item=42, A=0x7fffffffde70, size=6) at selection.c:28
#1  main () at selection.c:49

(gdb) list
23         if (hi < lo)
24             return 0;
25         int mid = (hi + lo) / 2;
26         if (A[mid] == item)
27             return 1;
28         if (item < A[mid])
29             hi = mid;
30         else
31             lo = mid;
32     }
```

```
(gdb) info locals
mid = 1
lo = 1
hi = 2
```

Second correction

```
Original:
if (item < A[mid])
    hi = mid;
else
    lo = mid;
```

```
Corrected:
if (item < A[mid])
    hi = mid - 1;
else
    lo = mid + 1;
```

Final output

```
$ ./selection
unsorted list:
    100
     45
     89
     27
    317
     17
list in order:
     17
     27
     45
     89
    100
    317
test if 42 is in the array...
NO
```

Task 3(B)(2) - Insertion Sort Error

Observed output

```
$ ./insertion
unsorted list:
  28
  4
 100
  9
 224
 111
  72
  53
list in order:
  8
  9
 28
 53
```

Faulty condition

```
while (j >= 0 && A[j] < A[j-1])
```

Correct condition

```
while (j > 0 && A[j] < A[j-1])
```

Explanation

When j becomes 0, the expression $A[j-1]$ becomes $A[-1]$, which refers to memory before the beginning of the array. This is an out-of-bounds access and causes undefined behavior. The invalid read/write explains the unexpected value 8 and the incorrect final list. The boundary check must ensure j is greater than 0 before $A[j-1]$ is accessed.

Task 3(C) - t4_msg.c and objdump

Makefile

```
CC = gcc
CFLAGS = -Wall

release:
■$(CC) $(CFLAGS) -o program t4_msg.c

debug:
■$(CC) $(CFLAGS) -g -o program t4_msg.c

clean:
■rm -f program
```

GDB evidence used to locate the bug

```
(gdb) break main
Breakpoint 1 at ...: file t4_msg.c, line 18.
(gdb) run
Breakpoint 1, main () at t4_msg.c:18
18     int i = 0;
(gdb) list
13     }
15     char* msg = "Jryy Qbar!!!\n";
17     int main() {
18         int i = 0;
19         printf("The secret message is: ");
20         while (i < len(msg)) printf("%c", rot13(msg[i++]));
22         return 0;
(gdb) print msg
$2 = 0x55555555556004 "Jryy Qbar!!!\n"
(gdb) print len(msg)
$3 = 0
(gdb) print msg[0]
$4 = 74 'J'
(gdb) print rot13(msg[0])
$5 = 87
```

Correction

```
Original:
int len(char *s) {
    int l = 0;
    while (*s) s++;
    return l;
}
```

```
Corrected:
int len(char *s) {
    int l = 0;
    while (*s) {
        s++;
        l++;
    }
    return l;
}
```

Final output

```
$ make debug
gcc -Wall -g -o program t4_msg.c
$ ./program
The secret message is: Well Done!!!
```

objdump commands and selected output

```
$ objdump --help
Usage: objdump <option(s)> <file(s)>
Display information from object <file(s)>.
-h Display the contents of the section headers
-d Display assembler contents of executable sections
-t Display the contents of the symbol table(s)
-g Display debug information in object file
-W Display the contents of DWARF debug sections

$ objdump -t program
program:      file format elf64-x86-64
SYMBOL TABLE:
```

```

0000000000001169 g      F .text 000000000000002e      len
0000000000001197 g      F .text 000000000000007e      rot13
0000000000001215 g      F .text 000000000000006d      main
0000000000004010 g      O .data 0000000000000008      msg

$ objdump -h program
Sections:
15 .text
17 .rodata
24 .data
25 .bss
27 .debug_aranges
28 .debug_info
29 .debug_abbrev
30 .debug_line
31 .debug_str
32 .debug_line_str

$ objdump -d program
Disassembly of section .text:
0000000000001169 <len>:
0000000000001197 <rot13>:
0000000000001215 <main>:

```

GDB location information

The program was compiled with -g. The resulting executable contains debugging information that lets GDB associate executable addresses with source files, line numbers, functions, variables and types. The `objdump -t` output also shows symbol addresses, while `objdump -h` shows debug sections such as `.debug_info` and `.debug_line`.

Final Submission Structure

```
lab3/
|-- task1/
|   |-- kohli.jpg
|   |-- task1_lab3_jp2a.txt
|-- task2_A/
|   |-- linkedlist.c
|   |-- linkedlist.h
|   |-- use_ll_2.c
|   |-- use_linkedlist.c
|   |-- Makefile
|-- task2_B/
|   |-- main.c
|   |-- addition.c
|   |-- subtraction.c
|   |-- multiplication.c
|   |-- division.c
|   |-- operations.h
|   |-- README.txt
|   |-- Makefile
|-- task3_A/
|   |-- L3.c
|   |-- main.c
|   |-- gdb.txt
|-- task3_B/
|   |-- selection.c
|   |-- insertion.c
|-- task3_C/
|   |-- t4_msg.c
|   |-- Makefile
|-- lab3_cs26mt010.pdf
```

Submission note

The assignment requires the report to be named lab3_rollno and stored in the lab3 folder. For roll number cs26mt010, the report filename is lab3_cs26mt010.pdf. The separate task/subtask folders are also required.